

Sincronizzazione e inter-process communication (IPC)

Antonio Pecchia

antonio.pecchia@unisannio.it

Dipartimento di Ingegneria
Laurea in Ingegneria Informatica
Sistemi Operativi



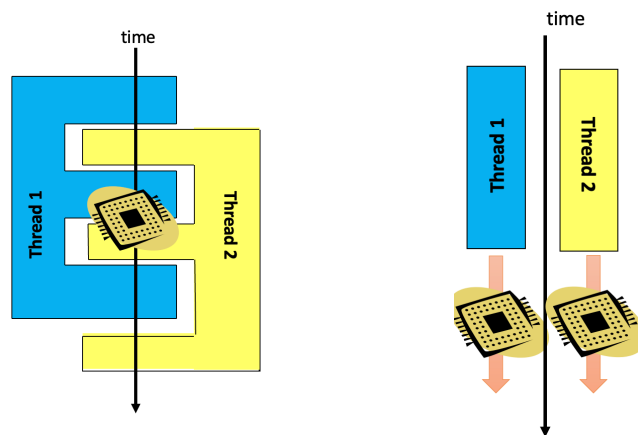
UNIVERSITÀ DEGLI STUDI
DEL SANNIO Benevento

Introduzione

- Fino ad ora: processi in "**isolamento**";
- Nella pratica, è molto comune che i processi **interagiscano** tra di essi:
 - condivisione di dati;
 - velocizzazione delle elaborazioni;
 - bilanciamento del carico;
 - modularità;
 - ...

Programmazione concorrente

- Insieme delle **metodologie** e **strumenti** necessarie per fornire il supporto all'esecuzione di applicazioni software come un insieme di attività svolte "simultaneamente".



Programmazione concorrente

- Primitive per **definire** *attività* indipendenti (processi, threads);
- Primitive per la **comunicazione** e **sincronizzazione** tra attività eseguite in modo concorrente* (concorrenza non significa necessariamente parallelismo).

***processi concorrenti**: processi la cui esecuzione si sovrappone nel tempo.

Lo scenario

- UNIX/Linux IPC facilities for **communication** and **synchronization** ("*The Linux Programming Interface*").

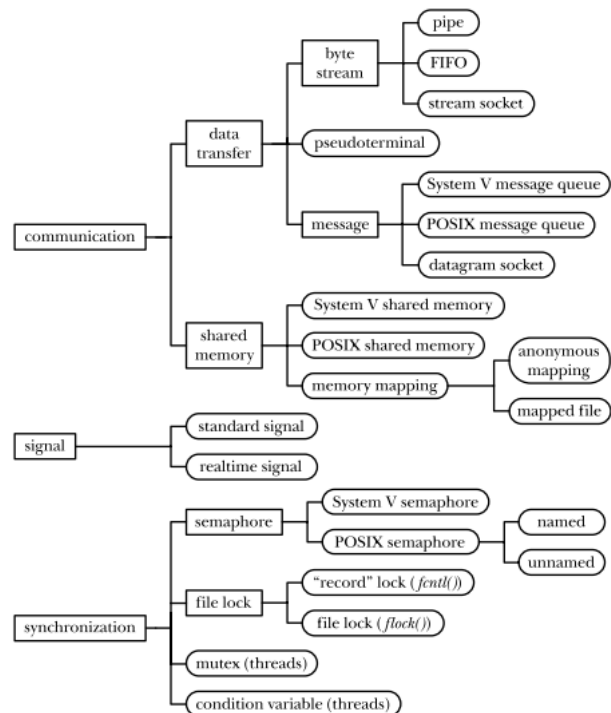


Figure 43-1: A taxonomy of UNIX IPC facilities

Race condition

- Una situazione in cui due o più processi leggono o scrivono dati condivisi e il risultato finale dipende dai tempi precisi in cui vengono eseguiti.

Processi concorrenti

- Processi **indipendenti**:
 - due processi P1 e P2 sono **indipendenti** se l'esecuzione di P1 non è influenzata da P2, e viceversa;
- Processi **interagenti**:
 - due processi P1 e P2 sono **interagenti** se l'esecuzione di P1 è influenzata da P2, e viceversa.

Tipi di interazione

- **Competizione**: per l'uso di risorse comuni che **non** possono essere utilizzate contemporaneamente;
- **Cooperazione**: nell'eseguire un'attività comune mediante scambio di informazioni;
- **Interferenza**: dovuta a competizione tra processi per uso non autorizzato di risorse comuni, oppure ad un *erronea soluzione* di problemi di competizione e di cooperazione.

Sincronizzazione

- ❑ Nei casi citati, è necessario imporre dei vincoli nella esecuzione delle operazioni dei processi;
- ❑ Vincoli per la sincronizzazione:
 - **competizione**: un solo processo alla volta deve avere accesso alla risorsa comune (*sincronizzazione indiretta o implicita*);
 - **cooperazione**: le operazioni eseguite dai processi concorrenti devono seguire una sequenza prefissata (*sincronizzazione diretta o esplicita*).

REGIONE CRITICA E MUTUA ESCLUSIONE

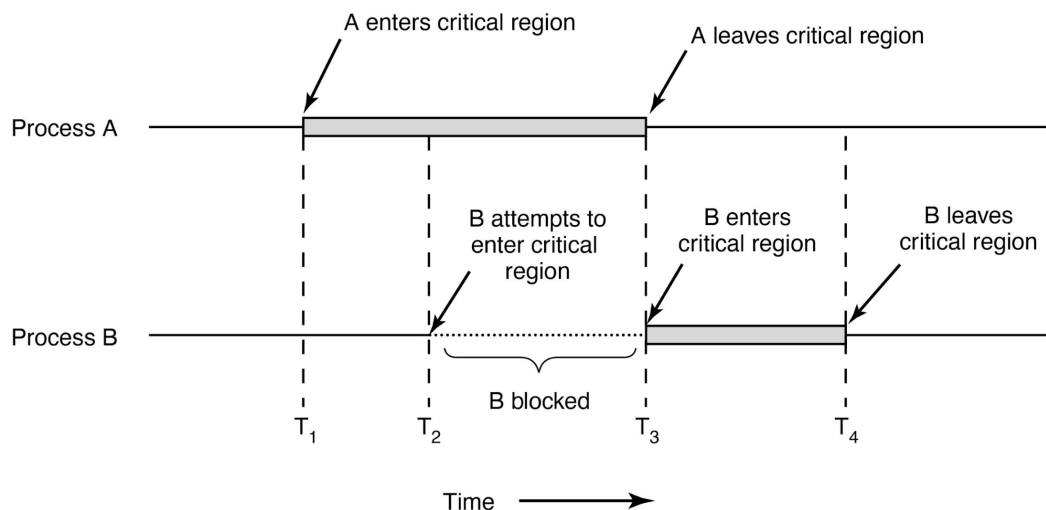
Cap 2: par. 2.4.1, 2.4.2, 2.4.3 (disabilitare gli interrupt, variabili lock e istruzione TSL), 2.4.4 (inversione della priorità)

Problema della mutua esclusione

- ❑ **Scenario:** due (o più) processi vogliono utilizzare una **risorsa** (per es., memoria o file condiviso, una stampante, etc.) ad uso *esclusivo*, denominata **risorsa critica**.
- ❑ La parte di programma che utilizza la risorsa è denominata **regione critica**.

obiettivo: fare in modo che *non* ci siano mai due processi che si trovano nelle loro regioni critiche allo stesso momento

Mutua esclusione con le regioni critiche



Condizioni

1. non devono mai esserci due processi contemporaneamente nelle loro regioni critiche;
2. non può essere fatta alcuna supposizione sulle velocità o sul numero delle CPU;
3. nessun processo in esecuzione al di fuori della sua regione critica può bloccare altri processi;
4. nessun processo dovrebbe restare per sempre in attesa di entrare nella sua regione critica.

Due (non) soluzioni

- ❑ **Disabilitare** gli interrupt;
- ❑ Utilizzo di una **variabile condivisa (lock)** inizializzata a 0:
 - quando un processo vuole *entrare* nella sezione critica, verifica il valore del lock:
 - se lock=0, posiziona lock a 1 ed entra nella sezione critica;
 - se lock=1, aspetta che diventi 0;
 - **problema**: lettura-scrittura del lock sono **divisibili**!

Istruzione TSL

- ❑ Risoluzione **via hardware** del problema delle variabili lock:
 - **istruzione macchina** TSL (**Test and Set Lock**):

TSL RX, LOCK

- legge il contenuto di LOCK in RX e salva un valore non zero in LOCK *in un solo ciclo**, in maniera indivisibile.

*la CPU che esegue TSL blocca il bus di memoria.

Istruzione TSL

- Istruzioni di entrata-uscita da una regione critica con l'istruzione TSL*:

```
enter_region:
    TSL REGISTER, LOCK | copia il lock nel registro e lo imposta a 1
    CMP REGISTER, #0   | il lock era zero?
    JNE enter_region   | se non era zero, il lock era stato impostato, per cui esegui il ciclo
    RET                | torna al chiamante; si è entrati nella regione critica

leave_region:
    MOVE LOCK, #0      | memorizza 0 in lock
    RET                | torna al chiamante
```

*istruzione alternativa: **XCHG** (per es., **scambio** di un registro e una parola di memoria in maniera atomica).

Problemi della soluzione proposta

- Condizioni di attesa attiva (**busy wait**), con relativo spreco di CPU;
- Problema della **priority inversion**:
 - un processo a priorità **bassa** (low=L) è nella sezione critica;
 - un processo a priorità **alta** (high=H) che vuole entrare nella sezione critica diventa ready ...
 - ... cosa succede se lo scheduler assegna la CPU ad H?

SEMAFORI

PRODUTTORE-CONSUMATORE LETTORI-SCRITTORI

Cap 2: par. 2.4.5

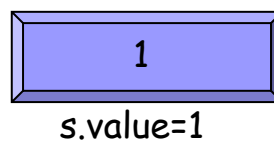
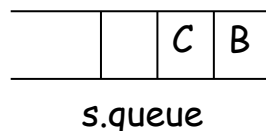
Semaforo

□ Tipo di dato caratterizzato da:

- un **valore intero**;
- una **lista** (coda) di processi in **attesa**;

```
typedef struct {  
    int value;  
    struct process *list;  
} semaphore;
```

semaforo



Semaforo: operazioni

- Inizializzazione ad un valore *non negativo*;
- **wait** e **signal****.

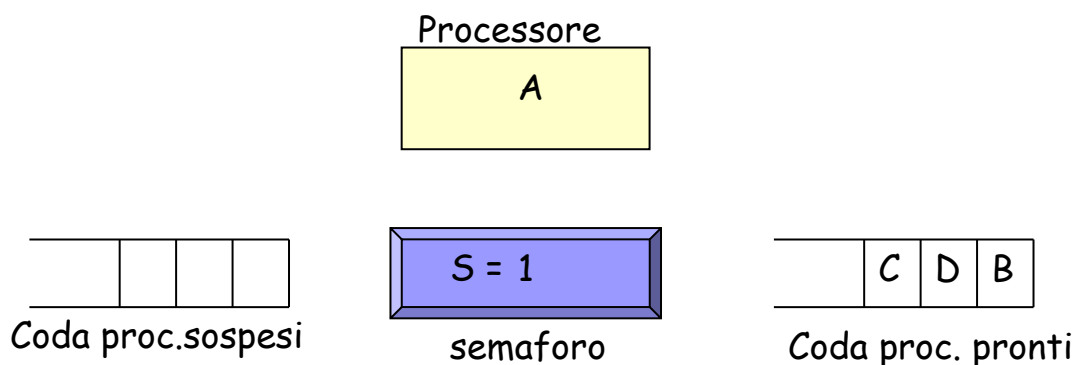
```
wait(semaphore *S) {  
    S->value--;  
    if (S->value < 0) {  
        add this process to S->list;  
        block();  
    }  
}
```

```
signal(semaphore *S) {  
    S->value++;  
    if (S->value <= 0) {  
        remove a process P from S->list;  
        wakeup(P);  
    }  
}
```

***block** sospende il processo che la invoca; **wakeup** pone in *pronto* un processo P bloccato.

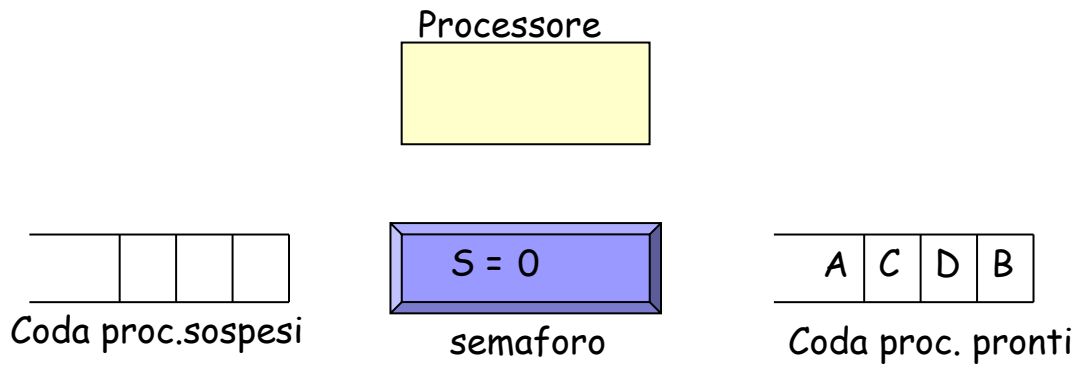
note anche come **down e **up** (libro di testo), oppure **P** (*proberen*, ossia provare) e **V** (*verhogen*, ossia incrementare) – E. W. Dijkstra (1965).

Semaforo: modello concettuale



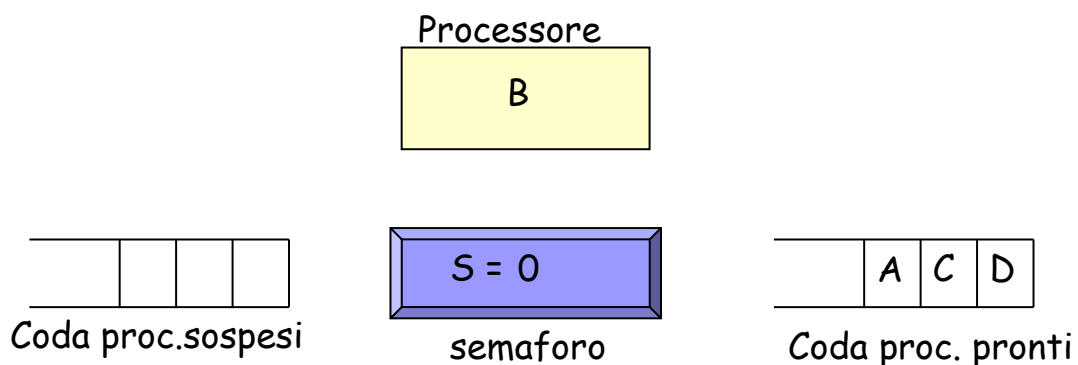
A esegue una wait

Semaforo: modello concettuale



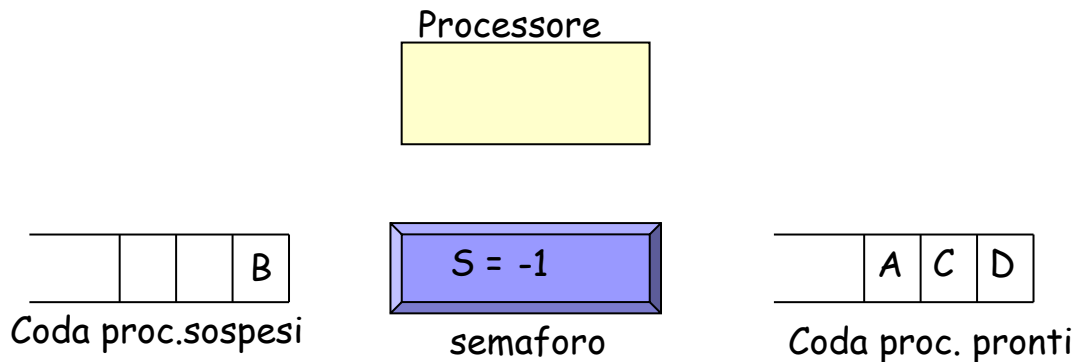
A esegue una wait

Semaforo: modello concettuale



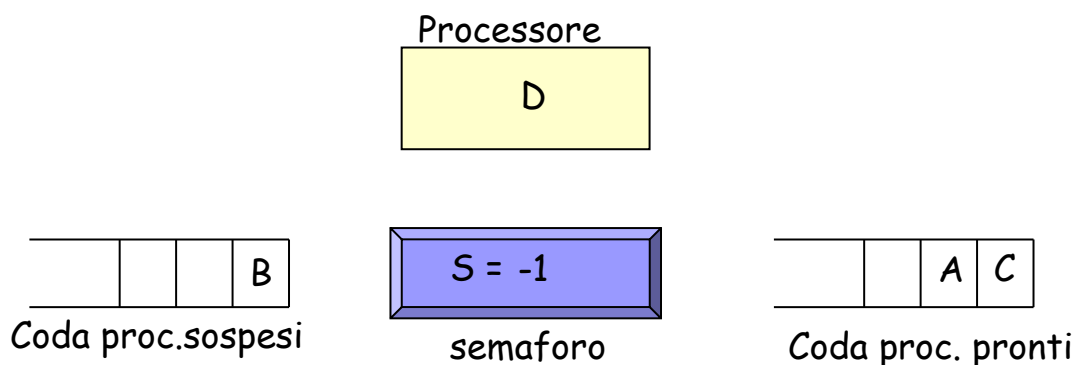
B esegue una wait

Semaforo: modello concettuale



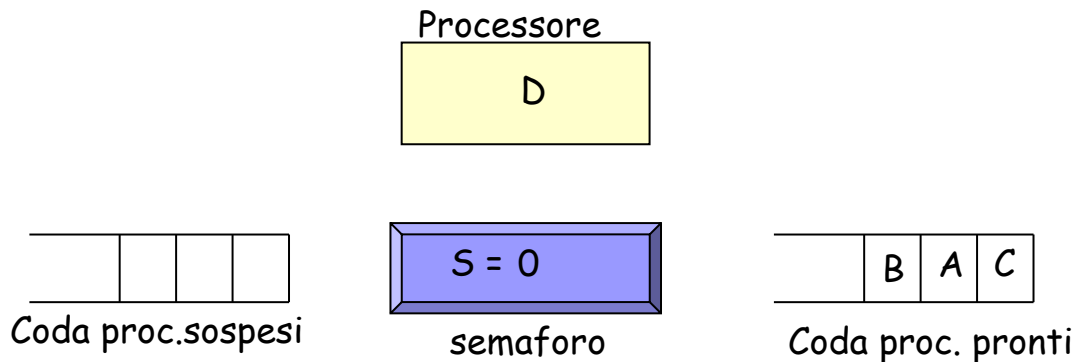
B esegue una wait

Semaforo: modello concettuale



D esegue una signal

Semaforo: modello concettuale



D esegue una signal

Semaforo: note

- L'aggiunta-eliminazione di processi dalla coda del semaforo *può* essere basata su strategia FIFO:
- È necessario garantire che due processi **non** possano eseguire contemporaneamente wait e signal sullo stesso semaforo: le operazioni sui semafori sono eseguite come singola azione **atomica** indivisibile:
 - per es., il SO disattiva brevemente gli interrupt mentre testa il semaforo (sistemi con *singola* unità di elaborazione) oppure tramite istruzioni TSL - XCHG;

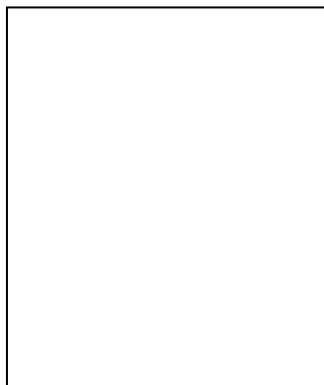
Mutua esclusione con semafori

- Si utilizza un semaforo inizializzato ad 1* (**semaforo binario**).

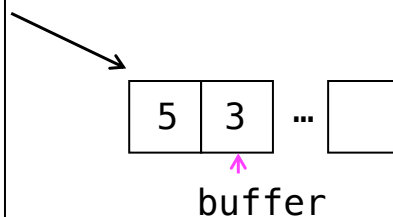
```
const int n = /* numero di processi */
semaphore mutex;
...
void P(int i) { /* il codice del processo i-mo*/
    ...
    wait(mutex);
    /* sezione critica */
    signal(mutex);
    ...
}
...
int main() {
    mutex.value = 1;
    /* esecuzione concorrente di P(1), ..., P(n) */
}
```

*talvolta denominato "mutex" (**mutual exclusion**); per **mutex** si intende tipicamente una versione "semplificata" di semaforo basata su una variabile che può trovarsi nei soli **2 stati** sbloccato / bloccato.

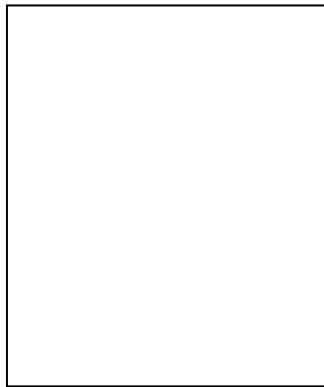
P1 (producer)



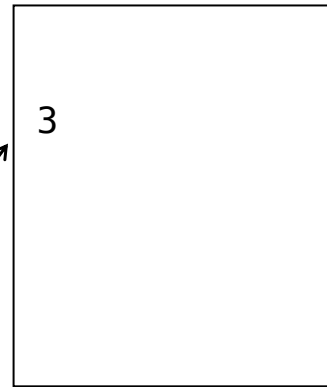
P2 (consumer)



P1 (producer)



P2 (consumer)



buffer

Cooperazione con semafori

- **Esempio** del problema del **produttore-consumatore***.

Due categorie di processi:

- **produttori**, che scrivono un messaggio in una risorsa condivisa;
- **consumatori**, che prelevano il messaggio dalla risorsa condivisa;

Vincoli:

- Il produttore non può produrre se il buffer è pieno;
- Il consumatore non può prelevare da un buffer vuoto.

* un altro problema famoso è quello dei **lettori-scrittori**.

Possibile soluzione

□ Utilizzo di tre semafori:

- il semaforo **mutex**, inizializzato a 1, serializza gli accessi al buffer condiviso;
- il semaforo **full**, inizializzato a 0, blocca il consumatore quando il buffer è **vuoto**;
- il semaforo **empty**, inizializzato a N (dove N è il numero di posti nel buffer), blocca il produttore quando il buffer è **pieno**.

```
#define N 100

typedef int sema;
sema mutex=1;
sema empty=N, full=0;

void producer(void) {
    int item;
    ...
    down(&empty);
    down(&mutex);
    insert_item(item);
    up(&mutex);
    up(&full);
    ...
}
```

```
void consumer(void) {
    int item;
    ...
    down(&full);
    down(&mutex);
    item = remove_item();
    up(&mutex);
    up(&empty);
    ...
}
```

Cooperazione con semafori

❑ Esempio del problema dei lettori-scrittori.

Due categorie di processi:

- **lettori**, che leggono un messaggio su di una risorsa condivisa;
- **scrittori**, che scrivono il messaggio in una risorsa condivisa;

Vincoli:

- i processi lettori possono accedere contemporaneamente alla risorsa;
- i processi scrittori hanno accesso esclusivo alla risorsa;
- lettori e scrittori si escludono mutuamente dall'uso della risorsa.

```
typedef int sema;  
sema mutex = 1;  
sema db = 1;  
int rc = 0;
```

```
void reader(){  
    ...  
    down (&mutex) ;  
    rc++;  
    if(rc==1) down (&db) ;  
    up (&mutex) ;  
    read_db() ;  
    down (&mutex) ;  
    rc--;  
    if(rc==0) up (&db) ;  
    up (&mutex) ;  
    ...  
}
```

```
void writer(){  
    ...  
  
    down (&db) ;  
    write_db() ;  
    up (&db) ;  
    ...  
}
```